# Project A: The $\pi$–$\lambda$ Theorem

## Lemmas 1–4, Helper Lemmas 1–5, and the final theorem

Math 7800 · Project A · Oral examination review

Fall 2026

# Proof roadmap: the paper proof of the $\pi$–$\lambda$ theorem

**Let $\mathcal{P}$ be a $\pi$-system and $L \supseteq \mathcal{P}$ a $\lambda$-system.**

1. $\lambda(\mathcal{P})$ is a $\lambda$-system containing $\mathcal{P}$; by minimality, $\lambda(\mathcal{P}) \subseteq L$.
2. Every $\lambda$-system contains $\varnothing$ and is closed under finite *disjoint*.
3. **Good sets.** For a fixed $A \in \lambda(\mathcal{P})$ define $G_A = \{B : A \cap B \in \lambda(\mathcal{P})\}$. Then $G_A$ is a $\lambda$-system.
4. **Two rounds.** If $A \in \mathcal{P}$, then $\mathcal{P} \subseteq G_A$, so $\lambda(\mathcal{P}) \subseteq G_A$. Repeat with $A \in \lambda(\mathcal{P})$ to obtain **intersection closure** of $\lambda(\mathcal{P})$.
5. A $\lambda$-system closed under intersections is a $\sigma$-field: disjointify an arbitrary sequence before taking its union.
6. Thus $\lambda(\mathcal{P})$ is a $\sigma$-field containing $\mathcal{P}$, and

$$\boxed{\sigma(\mathcal{P}) \subseteq \lambda(\mathcal{P}) \subseteq L.}$$

# Lemma 1: Generated $\lambda$-system is a $\lambda$-system

## Statement

For any class $\mathcal{A} \subseteq \mathcal{P}(\Omega)$, the generated class $\lambda(\mathcal{A})$ is a $\lambda$-system.

## Idea

It is the intersection of all $\lambda$-systems containing $\mathcal{A}$. Each of the three defining properties holds in every eligible $L$, hence in their intersection.

## Lean proof

```
lemma lambdaGen_isLambdaSys (A : Set (Set Ω)) :
    IsLambdaSys (lambdaGen A) := by
  refine ⟨?_, ?_, ?_⟩
  · exact mem_sInter.2 fun L hL => hL.1.univ_mem
  · intro B hB
    exact mem_sInter.2 fun L hL =>
      hL.1.compl_mem B ((mem_sInter.1 hB) L hL)
  · intro f hd hf
    exact mem_sInter.2 fun L hL =>
      hL.1.iUnion_mem f hd
        (fun n => mem_sInter.1 (hf n) L hL)
```

# Lemma 2: The generators belong to their generated class

## Statement

For every class $\mathcal{A}$, we have $\mathcal{A} \subseteq \lambda(\mathcal{A})$.

1. Fix $B \in \mathcal{A}$.
2. Every eligible $\lambda$-system $L$ contains $\mathcal{A}$, so $B \in L$.
3. Hence $B$ belongs to their intersection, $\lambda(\mathcal{A})$.

## Lean proof

```
lemma subset_lambdaGen (A : Set (Set Ω)) :
    A ⊆ lambdaGen A := by
  intro B hB
  exact mem_sInter.2 fun L hL => hL.2 hB
```

# Helper 1: Empty set belongs to a $\lambda$-system

## Mathematical statement

If $L$ is a $\lambda$-system, then $\varnothing \in L$.

1. By definition, $\Omega \in L$.
2. Complement closure gives $\Omega^c \in L$.
3. Since $\Omega^c = \varnothing$, conclude $\varnothing \in L$.

## Key Lean code

```
private lemma lambda_empty_mem
    (L : Set (Set Ω))
    -- L is a collection of subsets of Ω.

    (hL : IsLambdaSys L) :
    -- hL establishes that L is a λ-system.
    -- We may use:
    -- hL.univ_mem
    -- hL.compl_mem
    -- hL.iUnion_mem

    (∅ : Set Ω) ∈ L := by
  -- Goal: ∅ ∈ L.
have h_univ : (univ : Set Ω) ∈ L := hL.univ_mem
have h_compl : (univ : Set Ω)ᶜ ∈ L :=
  hL.compl_mem univ h_univ
simpa using h_compl
```

# Helper 2: Finite Disjoint Unions (Part 1)

## Statement

If $A, B \in L$ and $A \cap B = \varnothing$, then $A \cup B \in L$.

## Proof Overview

1. Construct a sequence with $f(0) = A$, $f(1) = B$, and $f(n) = \varnothing$ for $n \geq 2$.
2. Show every $f(n) \in L$, using Helper 1 for the empty sets.
3. Prove the sequence is pairwise disjoint.

## Lean Proof

```
private lemma lambda_union_mem_of_disjoint
    (L : Set (Set Ω))
    (hL : IsLambdaSys L)
    (A B : Set Ω)
    (hA : A ∈ L)
    (hB : B ∈ L)
    (hAB : Disjoint A B) :
    A ∪ B ∈ L := by

  let f : ℕ → Set Ω :=
    fun n =>
      if n = 0 then A
      else if n = 1 then B
      else ∅

  have hf_mem : ∀ n, f n ∈ L := by
    intro n
    by_cases h0 : n = 0
    case pos =>
      simp [f, h0, hA]
    case neg =>
      by_cases h1 : n = 1
      case pos =>
        simp [f, h1, hB]
      case neg =>
        simp [f, h0, h1, lambda_empty_mem L hL]

  have hf_disjoint : Pairwise (Disjoint on f) := by
    intro i j hij
    change Disjoint (f i) (f j)
    have hBA : Disjoint B A := hAB.symm
    by_cases hi0 : i = 0 <;>
      by_cases hi1 : i = 1 <;>
      by_cases hj0 : j = 0 <;>
      by_cases hj1 : j = 1 <;>
      simp_all [f]
```

# Helper 2: Finite Disjoint Unions (Part 2)

## Proof Overview

1. Prove $\bigcup_n f(n) = A \cup B$ by showing both set inclusions.
2. Apply the $\lambda$-system's countable disjoint-union property.
3. Rewrite the indexed union as $A \cup B$ to finish the proof.

## Lean Proof

```
have hforward : (⋃ n, f n) ⊆ A ∪ B := by
  intro x hx
  obtain ⟨n, hn⟩ := Set.mem_iUnion.mp hx
  change x ∈
    (if n = 0 then A else if n = 1 then B else ∅) at hn
  split_ifs at hn <;> simp_all

have hbackward : A ∪ B ⊆ (⋃ n, f n) := by
  intro x hx
  rcases hx with hx | hx
  case inl =>
    exact Set.mem_iUnion.mpr
      ⟨0, by simpa [f] using hx⟩
  case inr =>
    exact Set.mem_iUnion.mpr
      ⟨1, by simpa [f] using hx⟩

have h_union : (⋃ n, f n) = A ∪ B :=
  Set.Subset.antisymm hforward hbackward

have h_big_union : (⋃ n, f n) ∈ L :=
  hL.iUnion_mem f hf_disjoint hf_mem

rw [h_union] at h_big_union
exact h_big_union
```

# Helper 3: Relative Differences

## Statement

If $A, B \in L$ and $A \subseteq B$, then $B \setminus A \in L$.

## Proof Overview

1. Since $A \subseteq B$, the sets $A$ and $B^c$ are disjoint.
2. By complement closure, $B^c \in L$.
3. Apply Helper 2 to obtain $A \cup B^c \in L$.
4. Take the complement and use De Morgan's law: $(A \cup B^c)^c = B \setminus A$.
5. Therefore, $B \setminus A \in L$.

## Core Lean Proof

```
have hunion : A ∪ Bᶜ ∈ L :=
  lambda_union_mem_of_disjoint L hL A Bᶜ
    hA (hL.compl_mem B hB) hdisj

have hcompl : (A ∪ Bᶜ)ᶜ ∈ L :=
  hL.compl_mem (A ∪ Bᶜ) hunion

rw [heq] at hcompl
exact hcompl
```

# Helper 4: Minimality of the Generated $\lambda$-System

## Statement

If $L$ is a $\lambda$-system and $\mathcal{P} \subseteq L$, then $\lambda(\mathcal{P}) \subseteq L$.

## Proof Overview

1. Recall that $\lambda(\mathcal{P})$ is the intersection of all $\lambda$-systems containing $\mathcal{P}$.
2. Choose an arbitrary $E \in \lambda(\mathcal{P})$.
3. Since $L$ is a $\lambda$-system containing $\mathcal{P}$, it is one of the eligible systems.
4. By the definition of intersection, $E$ belongs to every eligible system, including $L$.
5. Therefore, $\lambda(\mathcal{P}) \subseteq L$.

## Core Lean Proof

```
intro E hE
  -- Introduce an arbitrary E ∈ lambdaGen P.
  -- Goal: E ∈ L.
exact (Set.mem_sInter.mp hE) L ⟨hL, hPL⟩
```

**Key Lean Tactic:**

`Set.mem_sInter.mp hE` converts membership in the intersection into membership in every eligible $\lambda$-system.

# Helper 5: The good-sets construction

## Statement

If $L$ is a $\lambda$-system and $D \in L$, define

$$G_D = \{B \subseteq \Omega : D \cap B \in L\}.$$

Then $G_D$ is a $\lambda$-system.

## Proof Overview

1. **Universal set:** $D \cap \Omega = D \in L$.
2. **Complements:** from $D \cap B \in L$, use Helper 3 to obtain

$$D \setminus (D \cap B) = D \cap B^c \in L.$$

3. **Disjoint unions:** for pairwise disjoint $f(n) \in G_D$, the sets $D \cap f(n)$ are pairwise disjoint and belong to $L$; then

$$\bigcup_n (D \cap f(n)) = D \cap \bigcup_n f(n) \in L.$$

# Helper 5: Main Lean Proof

```
refine {
  univ_mem := by
    change D ∩ univ ∈ L
    simpa only [inter_univ] using hD
  compl_mem := by
    intro B hB
    change D ∩ B ∈ L at hB
    change D ∩ Bᶜ ∈ L
    have hd : D \ (D ∩ B) ∈ L :=
      lambda_diff_mem L hL
        (D ∩ B) D hB hD inter_subset_left
    -- heq : D \ (D ∩ B) = D ∩ Bᶜ
    rw [heq] at hd
    exact hd
  iUnion_mem := by
    intro f hdisj hf
    change D ∩ (⋃ n, f n) ∈ L
    -- hd: Pairwise (Disjoint on (fun n => D ∩ f n))
    have hu : (⋃ n, D ∩ f n) ∈ L :=
      hL.iUnion_mem (fun n => D ∩ f n) hd hf
    -- heq : (⋃ n, D ∩ f n) = D ∩ (⋃ n, f n)
    rw [heq] at hu
    exact hu
}
```

# Lemma 3: Intersection Closure

## Statement

If $\mathcal{P}$ is a $\pi$-system, then

$$A, B \in \lambda(\mathcal{P}) \quad \Longrightarrow \quad A \cap B \in \lambda(\mathcal{P}).$$

## Main Proof Idea

We want to show that the generated $\lambda$-system is closed under intersections.
However, the $\pi$-system property only guarantees intersection closure for sets in $\mathcal{P}$.
Therefore, we extend intersection closure in two stages:

1. **Stage 1:** Extend the second argument from $\mathcal{P}$ to $\lambda(\mathcal{P})$:

$$D \in \mathcal{P}, \quad B \in \lambda(\mathcal{P}) \implies D \cap B \in \lambda(\mathcal{P}).$$

2. **Stage 2:** Extend the first argument from $\mathcal{P}$ to $\lambda(\mathcal{P})$:

$$A, B \in \lambda(\mathcal{P}) \implies A \cap B \in \lambda(\mathcal{P}).$$

# Lemma 3: Preparation

## Proof Overview

Before constructing the good-set families, we establish two facts:

1. $\lambda(\mathcal{P})$ is a $\lambda$-system.
2. The original $\pi$-system $\mathcal{P}$ is contained in $\lambda(\mathcal{P})$.

These facts allow us to apply Helper 5 to construct good-set families in both stages.

## Lean Proof

```
have hK : IsLambdaSys (lambdaGen P) :=
  lambdaGen_isLambdaSys P

have hPK : P ⊆ lambdaGen P :=
  subset_lambdaGen P
```

## Key Facts

- hK: $\lambda(\mathcal{P})$ is a $\lambda$-system.
- hPK: $\mathcal{P} \subseteq \lambda(\mathcal{P})$.

# Lemma 3: Stage 1 — Extend the Second Argument

## Goal

Prove that

$$D \in \mathcal{P}, \quad B \in \lambda(\mathcal{P}) \implies D \cap B \in \lambda(\mathcal{P}).$$

## Proof Overview

1. Fix $D \in \mathcal{P}$ and define
$$\mathcal{G}_D = \{B : D \cap B \in \lambda(\mathcal{P})\}.$$
2. Use Helper 5 to show that $\mathcal{G}_D$ is a $\lambda$-system.
3. Use the $\pi$-system property and $\mathcal{P} \subseteq \lambda(\mathcal{P})$ to prove $\mathcal{P} \subseteq \mathcal{G}_D$.
4. Apply Helper 4 (minimality) to obtain
$$\lambda(\mathcal{P}) \subseteq \mathcal{G}_D.$$

Therefore, $D \cap B \in \lambda(\mathcal{P})$ for every $B \in \lambda(\mathcal{P})$.

## Lean Proof

```
have hfirst :
    ∀ D ∈ P, ∀ B ∈ lambdaGen P,
      D ∩ B ∈ lambdaGen P := by
  intro D hD

  have hgood :
      IsLambdaSys {B | D ∩ B ∈ lambdaGen P} :=
    lambdaSys_good (lambdaGen P) hK D (hPK hD)

  have hcontains :
      P ⊆ {B | D ∩ B ∈ lambdaGen P} := by
    intro B hB
    exact hPK (hP D hD B hB)

  exact lambdaGen_min P
    {B | D ∩ B ∈ lambdaGen P}
    hgood hcontains
```

# Lemma 3: Stage 2 — Extend the First Argument

## Goal

Prove that

$$A, B \in \lambda(\mathcal{P}) \implies A \cap B \in \lambda(\mathcal{P}).$$

## Proof Overview

1. Fix $A \in \lambda(\mathcal{P})$ and define
$$\mathcal{G}_A = \{D : A \cap D \in \lambda(\mathcal{P})\}.$$
2. Use Helper 5 to show that $\mathcal{G}_A$ is a $\lambda$-system.
3. Use Stage 1 and intersection commutativity to prove $\mathcal{P} \subseteq \mathcal{G}_A$.
4. Apply Helper 4 again:
$$\lambda(\mathcal{P}) \subseteq \mathcal{G}_A.$$
5. Since $B \in \lambda(\mathcal{P})$, we obtain
$$A \cap B \in \lambda(\mathcal{P}).$$

## Lean Proof

```
intro A hA B hB

have hgood :
    IsLambdaSys {D | A ∩ D ∈ lambdaGen P} :=
  lambdaSys_good (lambdaGen P) hK A hA

have hcontains :
    P ⊆ {D | A ∩ D ∈ lambdaGen P} := by
  intro D hD
  change A ∩ D ∈ lambdaGen P
  simpa only [inter_comm] using hfirst D hD A hA

have hsub :
    lambdaGen P ⊆ {D | A ∩ D ∈ lambdaGen P} :=
  lambdaGen_min P
    {D | A ∩ D ∈ lambdaGen P}
    hgood hcontains

exact hsub hB
```

# Lemma 4: A $\lambda$-system closed under $\cap$ is a $\sigma$-field

## Statement

If $L$ is a $\lambda$-system and $A, B \in L \Rightarrow A \cap B \in L$, then $L$ is a $\sigma$-field.

## Overview

1. Universal-set and complement closure come directly from *hL*.
2. To prove arbitrary countable union closure, rewrite $\bigcup_n f(n)$ as the union of its *disjointed* sequence.
3. Each piece is built by repeated differences $t \setminus f(i) = t \cap f(i)^c$, still in $L$.
4. Apply `hL.iUnion_mem` to the now pairwise-disjoint pieces.

# Lemma 4: Lean Proof

## Disjointification and relative differences

```
lemma isSigmaField_of_lambda_pi (L : Set (Set Ω)) (hL : IsLambdaSys L)
    (hπ : ∀ A ∈ L, ∀ B ∈ L, A ∩ B ∈ L) : IsSigmaField L := by
  refine ⟨?_, ?_, ?_⟩ -- Break the proof into 3 parts
  · exact hL.univ_mem

  · exact hL.compl_mem

  · intro f hf
    -- ⋃ f equals the union of its disjointed version
    rw [← iUnion_disjointed]
    -- λ-systems contain disjoint unions; disjointed f is pairwise disjoint
    apply hL.iUnion_mem (disjointed f) (disjoint_disjointed f)
    -- still to show: each piece disjointed f n is in L
    intro n
    -- disjointed f n is f n with earlier f i's removed one at a time,
    -- so it is enough that L survives "t  t \ f i"
    -- Prove recursivly
    refine disjointedRec (p := (· ∈ L)) ?_ (hf n)
    intro t i ht
    -- t \ f i = t ∩ (f i)ᶜ, which is in L by hπ and compl_mem
    rw [sdiff_eq]
    exact hπ t ht (f i)ᶜ (hL.compl_mem (f i) (hf i))
```

# Final theorem: The $\pi$–$\lambda$ theorem

## Statement

If $\mathcal{P}$ is a $\pi$-system and $L$ is a $\lambda$-system with $\mathcal{P} \subseteq L$, then

$$\boxed{\sigma(\mathcal{P}) \subseteq L.}$$

## Proof Overview

1. Minimality gives $\lambda(\mathcal{P}) \subseteq L$.
2. Lemma 3 makes $\lambda(\mathcal{P})$ a $\pi$-system.
3. Lemma 4 make $\lambda(\mathcal{P})$ a $\sigma$-field.
4. Lemma 2 says $\mathcal{P} \subseteq \lambda(\mathcal{P})$; therefore $\sigma(\mathcal{P}) \subseteq \lambda(\mathcal{P})$.
5. Chain the inclusions to obtain $\sigma(\mathcal{P}) \subseteq L$.

# Final theorem: Lean proof

```
theorem pi_lambda (P L : Set (Set Ω))
    (hP : IsPiSys P) (hL : IsLambdaSys L)
    (hPL : P ⊆ L) : sigmaGen P ⊆ L := by
  have hGenLambda : lambdaGen P ⊆ L := by
    intro A hA
    unfold lambdaGen at hA
    exact Set.mem_sInter.mp hA L ⟨hL, hPL⟩

  have hGenPi : IsPiSys (lambdaGen P) := by
    exact lambdaGen_inter P hP

  have hGenSigma : IsSigmaField (lambdaGen P) := by
    exact isSigmaField_of_lambda_pi
      (lambdaGen P) (lambdaGen_isLambdaSys P) hGenPi

  have hSigmaSubset : sigmaGen P ⊆ lambdaGen P := by
    intro A hA
    unfold sigmaGen at hA
    exact Set.mem_sInter.mp hA (lambdaGen P)
      ⟨hGenSigma, subset_lambdaGen P⟩

  exact Set.Subset.trans hSigmaSubset hGenLambda
```